

Solar Paradox

(Software Engineering and System development)

ที่มาและความสำคัญ

ในปัจจุบันเกมแนว Puzzle ได้รับความนิยมอย่างต่อเนื่อง เนื่องจากเป็นสื่อที่สามารถพัฒนาทักษะการคิดเชิงตรรกะ (Logical Thinking) การแก้ปัญหาอย่างเป็นขั้นตอน (Problem Solving) และการมองระบบในภาพรวม (Systematic Thinking) ได้เป็นอย่างดี อย่างไรก็ตาม เกม Puzzle ส่วนใหญ่มักนำเสนอรูปแบบการเล่นในโครงสร้างพื้นที่เดียว (Single-layer World) ทำให้ผู้เล่นคาดเดาได้และขาดความท้าทายในระยะยาว

โครงการนี้จึงมีแนวคิดในการออกแบบเกม Puzzle ที่นำเสนอ “โลกซ้อนโลก” (Nested World System) โดยใช้ระบบสุริยะเป็นตัวแทนของลำดับชั้นของโลก ผู้เล่นสามารถมองเห็นโลกย่อยภายในวัตถุหรือพื้นที่หนึ่งได้ตั้งแต่ก่อนเข้าไปจริง ซึ่งช่วยกระตุ้นการคิดล่วงหน้า (Predictive Thinking) และการวางแผนการเดินอย่างรอบคอบ แนวคิดดังกล่าวช่วยเพิ่มมิติของการเรียนรู้ด้านโครงสร้างข้อมูลเชิงลำดับชั้น (Hierarchical Structure) ในรูปแบบที่เข้าใจง่ายและเป็นมิตรกับผู้เล่น

ในเชิงการดำเนินงานของโครงการ ระบบจะเริ่มจากการนำผู้เล่นเข้าสู่โลกหลัก (Main World) ซึ่งประกอบด้วยดาวเคราะห์หลายดวง แต่ละดวงทำหน้าที่เป็นด่านย่อย (Sub-world) เมื่อผู้เล่นเคลื่อนที่เข้าสู่พื้นที่ของดาว ระบบจะทำการซุ่มกล้องเข้าและเปลี่ยนขอบเขตการเล่นไปยังโลกย่อยโดยไม่เปลี่ยน Scene เพื่อให้การเล่นมีความต่อเนื่อง (Seamless Experience) ผู้เล่นสามารถแก้ปริศนาในโลกย่อยและกลับออกมายังโลกหลักได้ทันที

คุณสมบัติสำคัญของระบบ (Features)

- ระบบโลกซ้อนโลก (Nested World / Recursive Space) :** ระบบรองรับพื้นที่การเล่นที่สามารถซ้อนกันเป็นลำดับชั้น โดยโลกแต่ละระดับสามารถทำหน้าที่เป็นทั้งพื้นที่เล่นและวัตถุภายในโลกอื่น ผู้เล่นสามารถมองเห็นโครงสร้างของโลกย่อยล่วงหน้า และเข้าออกโลกย่อยได้อย่างต่อเนื่อง ช่วยสนับสนุนการวางแผนล่วงหน้าและการคิดเชิงโครงสร้างในการแก้ปริศนา
- ระบบกล้องซุ่มเข้า-ซุ่มออกตามระดับของโลก :** กล้องปรับระดับการซุ่มและตำแหน่งอัตโนมัติตามระดับของโลกที่ผู้เล่นอยู่ เพื่อสื่อถึงการเปลี่ยนขอบเขตของพื้นที่เล่นอย่างชัดเจน และช่วยให้ผู้เล่นเข้าใจโครงสร้างโลกซ้อนโลกโดยไม่ต้องเปลี่ยน Scene
- การเคลื่อนที่แบบ Grid-based Movement :** การเคลื่อนที่ของผู้เล่นและวัตถุเป็นแบบช่องตาราง (Grid) เพื่อเพิ่มความแม่นยำในการควบคุม ลดความคลุมเครือของการชนวัตถุ และเอื้อต่อการออกแบบปริศนาที่ต้องใช้การวางแผนอย่างเป็นขั้นตอน
- ระบบ Undo / Redo :** ระบบรองรับการย้อนกลับและทำซ้ำการเคลื่อนไหว โดยจัดเก็บสถานะของผู้เล่นและวัตถุในแต่ละขั้นตอน ช่วยให้ผู้เล่นสามารถทดลองแนวทางแก้ปริศนาได้โดยไม่สูญเสียความคืบหน้า และเป็นตัวอย่างของการจัดการ State ในระบบเกม
- สถาปัตยกรรมแบบ Single Scene (Single Scene Architecture) :** เกมทำงานภายใน Scene เดียว โดยใช้การจัดการข้อมูล วัตถุ และกล้องแทนการโหลดฉากใหม่ เพื่อรักษาความต่อเนื่องของประสบการณ์การเล่น และรองรับการซ้อนโลกหลายระดับได้อย่างมีประสิทธิภาพ

ประโยชน์ที่จะได้รับ

1. **ด้านผู้ใช้งาน :** ผู้เล่นได้รับการพัฒนาทักษะการคิดเชิงตรรกะ การวางแผนล่วงหน้า และการคิดแบบหลายชั้น ผ่านการแก้ปริศนาเชิงโครงสร้างที่แตกต่างจากเกม Puzzle ทั่วไป
2. **ด้านผู้พัฒนาและนักศึกษา :** โครงการนี้เป็นกรณีศึกษาการประยุกต์ใช้แนวคิดด้านวิทยาการคอมพิวเตอร์ เช่น Recursion และ State Management กับการพัฒนาเกมจริง ช่วยเสริมทักษะด้านการออกแบบระบบและการใช้ Unity Engine
3. **ด้านการศึกษาและการต่อยอด :** สามารถนำไปใช้เป็นสื่อการเรียนรู้เชิงโต้ตอบด้านตรรกะและการคิดเชิงระบบ รวมถึงต่อยอดเป็นผลงานสร้างสรรค์หรือพัฒนาเป็นผลิตภัณฑ์เกมในอนาคต

เงื่อนไขของระบบ

1. ผู้ใช้งานต้องมีอุปกรณ์ที่สามารถรองรับการทำงานของเกมที่พัฒนาด้วย Unity Engine เช่น เครื่องคอมพิวเตอร์หรือแล็ปท็อป
2. ระบบต้องทำงานภายใน Unity Engine เวอร์ชันที่รองรับการพัฒนาเกมแบบ 2 มิติ (2D)
3. มีการออกแบบโครงสร้างของโลก (World Structure) และระบบ Grid อย่างเป็นระบบและชัดเจน
4. ผู้พัฒนาต้องมีความรู้พื้นฐานด้านการเขียนโปรแกรมภาษา C# และแนวคิดเชิงวัตถุ (Object-Oriented Programming)
5. ระบบต้องไม่มีการเปลี่ยน Scene ระหว่างการเล่น โดยใช้การจัดการตำแหน่งและการซูมของกล้องเพื่อรองรับแนวคิดโลกซ้อนโลก (Recursive World)

เทคโนโลยีที่คาดว่าจะใช้ หรือเกี่ยวข้อง

1. Unity Game Engine (2D)
2. ภาษา C# สำหรับพัฒนาเกม
3. Orthographic Camera สำหรับควบคุมมุมมองและการซูม
4. Grid-based System Design สำหรับการเคลื่อนที่และการออกแบบด่าน
5. Object-Oriented Programming (OOP)
6. Coroutine สำหรับควบคุมลำดับเหตุการณ์ เช่น การซูมเข้า-ออก และการเปลี่ยนระดับของโลก

เอกสารอ้างอิง

1. [1] Unity Technologies. *Unity User Manual*. Available at: <https://docs.unity3d.com>
2. [2] Unity Technologies. *Orthographic Camera*. Available at: <https://docs.unity3d.com/Manual/class-Camera.html>
3. [4] Unity Technologies. *Tilemap System*. Available at: <https://docs.unity3d.com/Manual/Tilemap.html>
4. [5] Unity Technologies. *Unity Scripting API – C#*. Available at: <https://docs.unity3d.com/ScriptReference>